MCMP117 · Week 12 · Course Notes

Fundamentals of Operating Systems

Text Processing, Redirection & Pipes

Small programs, connected by the kernel, answer real questions about real data.

Instructor: Asst. Prof. Dr. (Ph.D.) Hasan Oğuz
Head of the Electronical Communication Technology Programme, Vocational School
Istanbul Okan University

What you should be able to do after this week

1. Explain standard input, standard output and standard error as the file descriptors 0, 1 and 2, and connect them to the `write(1, …)` system call of Weeks 2–3.
2. Use the redirections >, >>, <, 2>, 2>&1, &> and `/dev/null`, and predict the effect of their order.
3. Explain how the shell implements redirection and pipes with `fork()`, `open()`, `pipe()`, `dup2()` and `exec()`, and read the corresponding `strace` output.
4. Use the filters `grep` (with `-i -n -v -c -w -r` and simple regular expressions), `sort`, `uniq`, `cut`, `wc`, `tr`, `head`, `tail` and `tee`.
5. Search the directory tree with `find` and act on the results with `-exec` or `xargs`.
6. Answer questions about log files and CSV data with one-line pipelines, built one stage at a time.

Contents

1 From Week 11 to Week 12 1

2 Standard Streams and File Descriptors 2

3 Redirection 4

4 Pipes 6

5 Searching Lines: `grep` 9

6 Sorting and Counting: `sort` and `uniq` 11

7 Columns, Counts and Characters: `cut`, `wc`, `tr` 13

8 More Tools: `head`, `tail`, `tee`, `find`, `xargs` 14

9 Real Analyses with One-Line Pipelines 16

10 Command Reference 19

11 Common Errors and What They Mean 19

12 Lab Sheet 21

13 Summary 22

14 Glossary 22

15 Try It Yourself 23

16 Review Questions 23

17 Lab Sheet Solutions 24

18 Looking Ahead: Week 13 25

1 From Week 11 to Week 12

Last week we looked at files from the outside: who owns them (`id`, `/etc/passwd`), what the mode string `-rwxr-x---` means, and how `chmod` changes it, symbolically (`u+x`) or in octal (754: $7 = 4+2+1$ for the owner, $5 = 4+1$ for the group, 4 for others). This week we look *inside* files. Linux keeps almost everything — configuration, logs, data exports — as plain text, one record per line. A small set of tools that read lines, filter them and write lines can therefore answer a surprising range of questions. The glue that connects these tools is provided by the operating system: **redirection** and **pipes**. Both rest on one idea you have already met, the **file descriptor**.

How to follow these notes. All examples run in the practice folder `~/lab`. Re-create it before you start, so that your files are identical to ours:

```
$ bash mcmp117_lab_setup.sh
$ cd ~/lab
```

Every session shown here is real output, captured as an ordinary user with the scripts `code/demo_*.sh` (the output files are `code/out_*.txt`). On your machine, dates, process numbers (PIDs), terminal names (`/dev/pts/1`) and user names will differ. The order of `ls` and `sort` output can also depend on the *locale* (language settings); we used `LC_ALL=C.UTF-8`, in which upper-case letters sort before lower-case ones. Type `export LC_ALL=C.UTF-8` if your results are ordered differently.

2 Standard Streams and File Descriptors

2.1 Three open files from birth

In Week 7 we saw that a process does not refer to an open file by its name but by a small integer, the **file descriptor** (fd): an index into the process's *per-process open-file table*. When the shell starts a program, three descriptors are already open:

fd	Name	Used for	Default in a terminal
0	standard input (stdin)	data the program reads	the keyboard
1	standard output (stdout)	normal results	the terminal screen
2	standard error (stderr)	error and diagnostic messages	the terminal screen

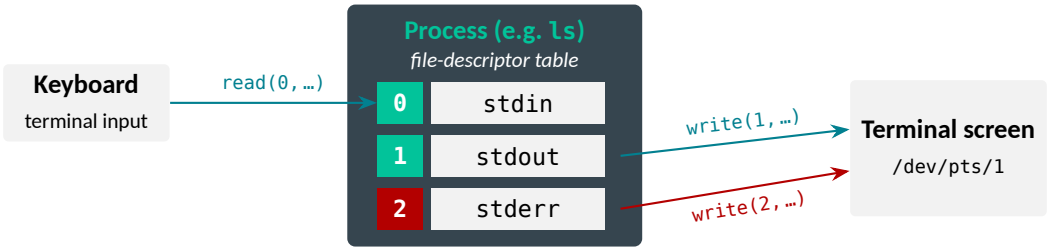


Figure 1: The three standard streams. Output and errors are two separate channels that, by default, end on the same screen.

Who opened them? The shell, or more precisely the chain of processes that started the shell. A child process created with `fork()` receives a copy of its parent's descriptor table, and `exec()` keeps it (Week 3). So when `bash` runs `ls`, `ls` inherits `bash`'s descriptors 0, 1, 2, all pointing to the terminal.

Why two output channels? Imagine `ls` saves its listing in a file and one of its arguments does not exist. If the error message were mixed into the listing, it would end up hidden in the file. Because errors travel on their own descriptor, they still reach the screen. This simple design decision is the reason the `2>` redirection exists.

2.2 You have seen this before: `write(1, …)`

In Week 2, `strace` showed that `printf("Hello…")` ends up as the system call `write(1, "Hello…", n)`, and in Week 3 we executed the same call by hand with the `syscall` instruction: the first parameter, 1, was standard output. Let us trace `ls` with one existing and one missing argument (`-o t1.txt` writes the trace into a file so that it does not mix with the output of `ls`):

```
student@okan-lab:~/lab$ strace -o t1.txt -e trace=write ls data nosuch
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat t1.txt
write(2, "ls: ", 4)                      = 4
write(2, "cannot access 'nosuch'", 22)   = 22
write(2, ": No such file or directory", 27) = 27
write(2, "\n", 1)                        = 1
write(1, "data:\n", 6)                   = 6
write(1, "cities.txt  numbers.txt  student"..., 38) = 38
```

```
+++ exited with 2 +++
```

The four pieces of the error message go out with `write(2, …)`, the listing with `write(1, …)`. The program `ls` never asks *where* descriptors 1 and 2 lead; that was decided before it started. If the shell makes descriptor 1 point to a file, every `write(1, …)` lands in the file. That is all redirection is.

A small surprise. On the terminal, `ls` printed `cities.txt  numbers.txt  students.csv` in columns, but the trace (and every file into which we redirect it, below) contains one name per line. `ls` checks with the library function `isatty(1)` whether descriptor 1 is a terminal and chooses its format accordingly. Most programs do not care; a few (`ls`, `grep --color=auto`) do.

2.3 Seeing the descriptors: `/proc/self/fd`

The `/proc` file system (Week 3) shows the kernel's view of every process. `/proc/self` is the process that is looking, and `/proc/self/fd` lists its descriptors as symbolic links. Running `ls -l /proc/self/fd` therefore shows the descriptors of `ls` itself:

```
student@okan-lab:~/lab$ ls -l /proc/self/fd
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15829/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd > fd.txt
student@okan-lab:~/lab$ cat fd.txt
total 0
lrwx------ 1 student student 64 Oct 12 08:48 0 -> /dev/pts/1
l-wx------ 1 student student 64 Oct 12 08:48 1 -> /home/student/lab/fd.txt
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15832/fd
student@okan-lab:~/lab$ ls -l /proc/self/fd < docs/syllabus.txt
total 0
lr-x------ 1 student student 64 Oct 12 08:48 0 -> /home/student/lab/docs/syllabus.txt
lrwx------ 1 student student 64 Oct 12 08:48 1 -> /dev/pts/1
lrwx------ 1 student student 64 Oct 12 08:48 2 -> /dev/pts/1
lr-x------ 1 student student 64 Oct 12 08:48 3 -> /proc/15838/fd
```

- Normally, 0, 1 and 2 all point to the terminal device `/dev/pts/1`. Descriptor 3 is the directory that `ls` is reading at that moment.
- With `> fd.txt`, descriptor 1 points to the file `/home/student/lab/fd.txt`; `ls` wrote its own listing into it. The mode `l-wx------` shows that the file is open for writing only.
- With `< docs/syllabus.txt`, descriptor 0 is the file, opened read-only (`lr-x------`).
- With `2> /dev/null` (see `code/out_streams.txt`), descriptor 2 points to `/dev/null`.

3 Redirection

3.1 Output: > and >>

```
student@okan-lab:~/lab$ echo "first line" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
student@okan-lab:~/lab$ echo "second line" >> notes.txt
student@okan-lab:~/lab$ cat notes.txt
first line
second line
student@okan-lab:~/lab$ echo "oops" > notes.txt
student@okan-lab:~/lab$ cat notes.txt
oops
```

`cmd > file` creates the file if necessary, *empties it* (truncates it to 0 bytes) and connects descriptor 1 to it. `cmd >> file` opens it for *appending*: new data goes to the end. The third `echo` above replaced both lines: the old content is gone, with no undo and no recycle bin (Week 10). Bash can protect you: after `set -o noclobber`, the shell refuses to overwrite an existing file with > (the operator >| forces it).

3.2 Input: <

```
student@okan-lab:~/lab$ wc -l logs/access.log
300 logs/access.log
student@okan-lab:~/lab$ wc -l < logs/access.log
300
```

With `wc -l logs/access.log`, `wc` opens the file itself and knows its name, so it prints the name. With `wc -l < logs/access.log`, the *shell* opens the file as descriptor 0 and `wc` just reads its standard input: it has no name to print. The bare number is handy in scripts (Week 14).

The filter convention

Most text tools are **filters**: they read the files named on the command line, or standard input if no file is named, and write their result to standard output. Error messages go to standard error. Because of this convention, any filter can be fed from a file, from the keyboard, or from another program — and its output can go to the screen, to a file, or to another program.

3.3 Errors: 2>, 2>&1, &> and /dev/null

The general form of a redirection is `n>file`, where n is a descriptor number; > alone means 1>, and < alone means 0<.

```
student@okan-lab:~/lab$ ls data nosuch > out.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ cat out.txt
data:
cities.txt
numbers.txt
students.csv
student@okan-lab:~/lab$ ls data nosuch 2> err.txt
data:
cities.txt  numbers.txt  students.csv
student@okan-lab:~/lab$ cat err.txt
ls: cannot access 'nosuch': No such file or directory
```

```
student@okan-lab:~/lab$ ls data nosuch > all.txt 2>&1
student@okan-lab:~/lab$ cat all.txt
ls: cannot access 'nosuch': No such file or directory
data:
cities.txt
numbers.txt
students.csv
```

```
student@okan-lab:~/lab$ ls data nosuch &> all.txt
student@okan-lab:~/lab$ ls data nosuch 2> /dev/null
data:
cities.txt  numbers.txt  students.csv
```

- `> out.txt` moves only standard output, so the error still appears on the screen.
- `2> err.txt` moves only standard error: the listing appears, the error is in the file.
- `> all.txt 2>&1` puts both in the file. `2>&1` reads "make descriptor 2 a copy of descriptor 1".
- `&> all.txt` is a bash shorthand for the same thing (it does not work in the simpler `sh`/`dash`).
- `2> /dev/null` throws the errors away. `/dev/null` is a *device file*: whatever is written to it disappears, and reading from it immediately returns end-of-file.

3.4 Order matters

Redirections are processed *from left to right*, and `2>&1` copies descriptor 1 *as it is at that moment*; it is not a permanent link. Compare:

```
student@okan-lab:~/lab$ ls data nosuch 2>&1 > out2.txt
ls: cannot access 'nosuch': No such file or directory
student@okan-lab:~/lab$ cat out2.txt
data:
cities.txt
numbers.txt
students.csv
```

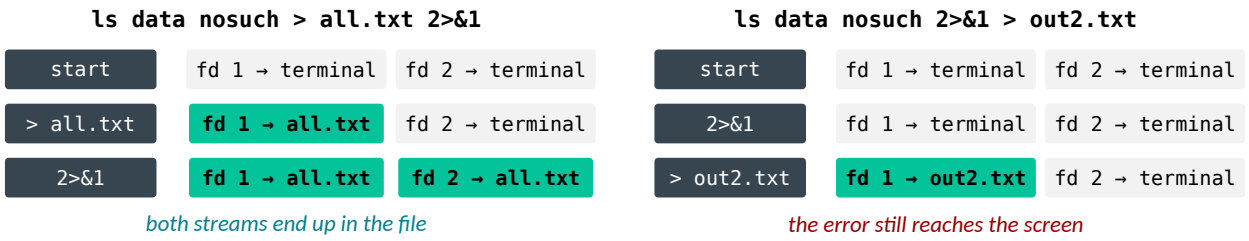


Figure 2: The descriptor table after each redirection, processed left to right.

3.5 How the shell does it

In Week 3 we asked why UNIX creates processes with two separate calls, `fork()` and `exec()`, and answered: between them, the child can prepare its environment, for example redirect its output. Here is the proof. We trace a non-interactive `bash` running `ls data > list.txt`, following children (`-f`) and showing only three system calls:

```
student@okan-lab:~/lab$ strace -f -o t2.txt -e trace=openat,dup2,execve bash -c "ls data >
    list.txt"
student@okan-lab:~/lab$ grep -E "list.txt|dup2|execve" t2.txt
15726 execve("/usr/bin/bash", ["bash", "-c", "ls data > list.txt"], 0x7fff5f5520e8 /* 23 vars
    */) = 0
15727 openat(AT_FDCWD, "list.txt", O_WRONLY|O_CREAT|O_TRUNC, 0666) = 3
15727 dup2(3, 1)                        = 1
15727 execve("/usr/bin/ls", ["ls", "data"], 0x55b277587230 /* 23 vars */) = 0
```

The first number on each line is the PID: 15726 is `bash`, 15727 its child (created with `clone()`, the Linux system call behind `fork()`, not traced here). The child

1. opens `list.txt` for writing; `O_CREAT` creates it, `O_TRUNC` empties it (that is the "> destroys the old content" of the previous section), and the kernel returns the lowest free descriptor, 3;
2. calls `dup2(3, 1)`: descriptor 1 becomes a copy of descriptor 3, so standard output now leads to the file (the shell then closes the spare descriptor 3);
3. calls `execve("/usr/bin/ls", …)`: the program is replaced by `ls`, but the descriptor table survives. `ls` writes to descriptor 1 — the file — and never knows it was redirected.

In the same way, >> opens with `O_APPEND` instead of `O_TRUNC`, < opens read-only and duplicates onto descriptor 0, and 2>&1 is simply `dup2(1, 2)`. Redirection is not a feature of `ls` or of any other command: it is done by the shell, with ordinary system calls.

4 Pipes

4.1 Connecting processes

Redirection connects a process to a *file*. A **pipe**, written `cmd1 | cmd2`, connects a process to another *process*: the standard output of `cmd1` becomes the standard input of `cmd2`. A chain of commands joined by pipes is a **pipeline**.

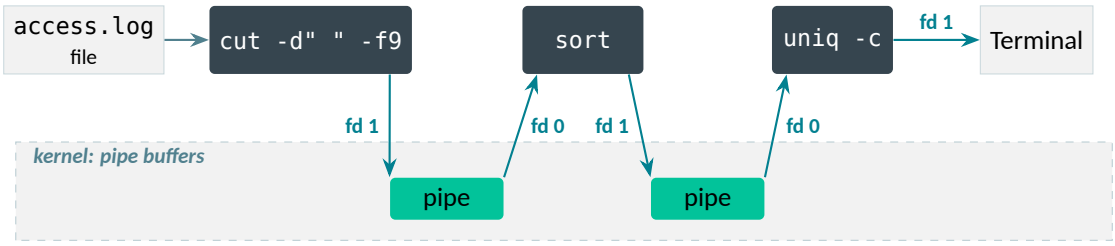


Figure 3: The pipeline `cut -d" " -f9 logs/access.log | sort | uniq -c`: three processes running at the same time, connected through two buffers inside the kernel. Standard error (fd 2) of every stage still goes to the terminal.

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c
    148 200
     20 301
     34 304
     17 403
     57 404
     24 500
```

`cut` extracts field 9 (the HTTP status code) from each line, `sort` brings equal codes together, `uniq -c` counts each group. No temporary files are created: the data flows through memory buffers owned by the kernel. Each stage is an independent process with its own PID (Week 3), and the shell starts all of them.

4.2 How the shell builds a pipe

A pipe is created with the system call `pipe()` (in modern Linux `pipe2()`), which returns *two* descriptors: one for reading and one for writing. Tracing a two-stage pipeline:

```
student@okan-lab:~/lab$ strace -f -o t3.txt -e trace=pipe2,clone,dup2,execve bash -c "grep
    ERROR logs/system.log | wc -l"
13
student@okan-lab:~/lab$ cat t3.txt
15736 execve("/usr/bin/bash", ["bash", "-c", "grep ERROR logs/system.log | wc "...], 0
    x7ffe43f1a528 /* 23 vars */) = 0
15736 pipe2([3, 4], 0)                  = 0
15736 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7f6367df1a10) = 15737
15736 clone(child_stack=NULL, flags=CLONE_CHILD_CLEARTID|CLONE_CHILD_SETTID|SIGCHLD,
    child_tidptr=0x7f6367df1a10) = 15738
15737 dup2(4, 1)                        = 1
15738 dup2(3, 0)                        = 0
15738 execve("/usr/bin/wc", ["wc", "-l"], 0x55c9a2ad4920 /* 23 vars */) = 0
15737 execve("/usr/bin/grep", ["grep", "ERROR", "logs/system.log"], 0x55c9a2ad4920 /* 23 vars
    */) = 0
```

1. `pipe2([3, 4], 0)`: `bash` (15736) gets a pipe: descriptor 3 is the read end, 4 the write end.
2. Two `clone()` calls (= `fork()`) create the children 15737 and 15738; both inherit descriptors 3 and 4.
3. The first child calls `dup2(4, 1)`: its standard output is now the write end. It then executes `grep`.
4. The second child calls `dup2(3, 0)`: its standard input is the read end. It executes `wc`.
5. Unused ends are closed (`close()` was not traced), and `bash` waits for both children (the remaining lines of the trace, omitted here, show the two exits and the `SIGCHLD` signal with which the kernel tells the parent that a child has terminated).

Notice that `wc` was executed *before* `grep` in this run: the two children run concurrently, and the scheduler (Week 4) decides who goes first. When two processes make system calls at the same moment, `strace` may print a call in two halves, marked `<unfinished ...>` and `<... resumed>`; that is normal.

A pipe is visible in `/proc` too: in `ls -l /proc/self/fd | cat`, the descriptor 1 of `ls` is shown as `1 -> pipe:[29942]`, a pipe identified by a number inside the kernel (full output in `code/out_pipes.txt`).

Redirection and pipes in four system calls

`pipe()` creates the channel, `fork()` creates the processes, `dup2()` plugs the channel (or a file opened with `open()`) into descriptor 0, 1 or 2, and `exec()` starts the program, which inherits the plumbing without knowing it.

4.3 A pipe is a bounded buffer (Week 5)

A pipe has a limited capacity: on Linux the default is 64 KiB $= 16 \times 4096 = 65\,536$ bytes (sixteen 4 KiB pages). The kernel enforces three rules:

- **Buffer full:** `write()` blocks — the writer is put to sleep until the reader has removed data.
- **Buffer empty:** `read()` blocks — the reader sleeps until data arrives.
- **No writer left and buffer empty:** `read()` returns 0, which means end-of-file; the reader finishes. **No reader left:** the writer receives the signal `SIGPIPE`, which terminates it.

This is exactly the *bounded-buffer* (producer–consumer) problem of Week 5, solved inside the kernel: the producer must wait when the buffer is full, the consumer when it is empty. Two experiments show the consequences:

```
student@okan-lab:~/lab$ time (sleep 2 | sleep 2)

real    0m2.005s
user    0m0.003s
sys     0m0.002s
student@okan-lab:~/lab$ yes | head -n 3
y
y
y
student@okan-lab:~/lab$ yes "MCMP117" | head -n 100000 | wc -l
100000
```

`sleep 2 | sleep 2` takes 2.0 s, not 4 s: the stages run *concurrently*. `yes` prints y forever, yet the pipeline stops: after three lines head exits, the pipe has no reader, and the next `write()` of `yes` triggers `SIGPIPE`. (Signals are a topic of Week 13.) The last pipeline pushes $100\,000$ lines, about 800 kB, through two 64 KiB buffers without any problem: data is consumed as fast as it is produced.

> **Pipes and the UNIX philosophy**
>
> In a 1964 memo, Doug McIlroy at Bell Labs wrote that programs should be connected "like garden hose — screw in another segment when it becomes necessary to massage data in another way". Pipes appeared in UNIX Version 3 in 1973, when Ken Thompson added the `pipe()` system call and the `|` notation to the shell. McIlroy later summarised the resulting style: write programs that do one thing and do it well; write programs to work together; write programs to handle text streams, because that is a universal interface. `grep` itself is named after the command `g/re/p` ("global regular expression print") of the ed text editor.

5 Searching Lines: `grep`

`grep [options] PATTERN [files]` prints the lines that contain the pattern.

```
student@okan-lab:~/lab$ grep ERROR logs/system.log | head -n 3
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:56:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:07:00 ERROR failed to mount /dev/sdb1
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep -i "TALK" docs/linux_quotes.txt
```

```
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -n "backup" logs/system.log | head -n 4
6:2026-10-12 09:55:00 INFO  backup finished
7:2026-10-12 09:06:00 INFO  backup started
10:2026-10-12 09:39:00 INFO  backup finished
29:2026-10-12 10:08:00 INFO  backup started
student@okan-lab:~/lab$ grep -v INFO logs/system.log | head -n 4
2026-10-12 09:11:00 WARN  high memory usage
2026-10-12 09:22:00 WARN  high memory usage
2026-10-12 09:12:00 ERROR failed to mount /dev/sdb1
2026-10-12 09:23:00 WARN  temperature 71C
student@okan-lab:~/lab$ grep -vc INFO logs/system.log
41
```

Option	Meaning
`-i`	ignore the difference between upper and lower case
`-n`	prefix each line with its line number
`-v`	invert: print the lines that do *not* match
`-c`	print only the number of matching *lines* (not occurrences)
`-w`	the match must be a whole word
`-r`	search all files below a directory (recursively)
`-l`	print only the names of files that contain a match

Options can be combined: `grep -vc INFO` counts the non-INFO lines, 41. (Later we will see that these are 13 ERROR and 28 WARN lines, and $13 + 28 = 41$.) The minutes in `system.log` are not in order within an hour; this is generated practice data, while real logs are in time order.

5.1 Whole words and whole directories

```
student@okan-lab:~/lab$ grep "Week 1" docs/syllabus.txt
Week 1  Introduction and history
Week 10 File and directory operations
Week 11 Permissions, ownership and users
Week 12 Text processing and redirection
Week 13 Processes and job control
Week 14 Introduction to shell scripting
Week 15 Final exam
student@okan-lab:~/lab$ grep -w "Week 1" docs/syllabus.txt
Week 1  Introduction and history
student@okan-lab:~/lab$ grep -r "Project" projects
projects/beta/README.md:# Project beta
projects/alpha/README.md:# Project alpha
```

The pattern `Week 1` also occurs inside `Week 10` … `Week 15`. With `-w`, the match must not be preceded or followed by a letter, digit or underscore. With `-r`, `grep` searches every file below `projects` and prefixes each line with the file name; the files appear in directory order, not alphabetically.

5.2 Simple regular expressions

The pattern of `grep` is a **regular expression** (regex): a small language for describing sets of strings. Most characters match themselves; a few have a special meaning.

Symbol	Matches	Example	Matches, e.g.
`^`	the start of the line	`^Week`	lines beginning with `Week`
`$`	the end of the line	`s$`	lines ending in `s`
`.`	any *one* character	`c.de`	`code`, `cxde`
`[ ]`	one character from a set or range	`[0-9]`, `[aeiou]`	`7`, `e`
`*`	the preceding item, 0 or more times	`[0-9]*`	empty, `4`, `2026`
`\.`	a literal dot (`\` removes the special meaning)	`5\.38`	`5.38` only

```
student@okan-lab:~/lab$ grep "^Week [0-9] " docs/syllabus.txt
Week 1  Introduction and history
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
...
student@okan-lab:~/lab$ grep "s$" docs/syllabus.txt
Week 2  Definitions, goals, components, structures
Week 3  OS services, system calls, processes
Week 4  Threads and CPU scheduling basics
...
student@okan-lab:~/lab$ grep "c.de" docs/linux_quotes.txt
Talk is cheap. Show me the code.
student@okan-lab:~/lab$ grep -c "5.38" logs/access.log
39
student@okan-lab:~/lab$ grep -c "5\.38" logs/access.log
37
student@okan-lab:~/lab$ grep "5.38" logs/access.log | grep -v "5\.38"
10.0.6.45 - - [12/Oct/2026:15:53:47 +0300] "GET /courses/mcmp117 HTTP/1.1" 200 5438
10.0.2.17 - - [12/Oct/2026:15:38:02 +0300] "GET /login HTTP/1.1" 200 1357
student@okan-lab:~/lab$ grep ",ECT,[0-9]*,9[0-9]$" data/students.csv
2026001,Elif Yilmaz,ECT,96,99
2026015,Deniz Celik,ECT,95,96
student@okan-lab:~/lab$ grep "^2026-10-12 1[0-1]:.*ERROR" logs/system.log
2026-10-12 10:40:00 ERROR failed to mount /dev/sdb1
2026-10-12 10:35:00 ERROR failed to mount /dev/sdb1
2026-10-12 10:30:00 ERROR failed to mount /dev/sdb1
2026-10-12 10:03:00 ERROR failed to mount /dev/sdb1
```

- `^Week [0-9] ` (note the final space) matches `Week`, a space, one digit and a space at the start of the line: the single-digit weeks 1–9.
- The unescaped dot is a classic trap. Counting the requests of the IP address `10.0.5.38` with the pattern `5.38` gives 39, but the correct answer is 37: the dot also matched the `:` in the time `15:38`

and the digit 4 in the byte count 5438. The last `grep` pipeline shows exactly those two extra lines.

- `,ECT,[0-9]*,9[0-9]$` finds ECT students whose *final* grade (the last field) is between 90 and 99.
- `.*` means "any number of any characters": `^2026-10-12 1[0-1]:.*ERROR` finds ERROR lines between 10:00 and 11:59.

Regular expressions are not wildcards

In the shell (Week 10), `*` means "any string" and `?` "any character". In a regular expression, `*` means "repeat the previous item" and `.` means "any character". Always put the pattern in quotes, so that the shell does not try to expand `*`, `[ ]` or spaces before `grep` sees them. (`grep -E` enables *extended* regular expressions with `+`, `?`, `|` and groups; we do not need them this week.)

6 Sorting and Counting: `sort` and `uniq`

```
student@okan-lab:~/lab$ sort data/cities.txt | head -n 4
Ankara
Ankara
Antalya
Antalya
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort | head -n 3
100
35
36
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f5 | sort -n | head -n 3
35
36
36
student@okan-lab:~/lab$ sort -t, -k5 -nr data/students.csv | head -n 3
2026010,Onur Dogan,ME,73,100
2026001,Elif Yilmaz,ECT,96,99
2026020,Emre Dogan,ME,64,98
```

Without options, `sort` compares lines *character by character*, so `100` comes before `35` (the character `1` is smaller than `3`). With `-n` it compares numbers. Main options:

`-n`	numeric order ($35 < 100$)
`-r`	reverse order (largest first)
`-t,`	the field separator is a comma (default: blanks)
`-k5`	sort by field 5 (from field 5 to the end of the line; `-k5,5` means field 5 only)
`-u`	output each distinct line only once
`-o file`	write the result to *file*; safe even if *file* is the input

The command `tail -n +2` prints a file from line 2 onwards, skipping the CSV header `id,name,department,midterm,final`; see Section 11 for what happens if you forget it.

`uniq` removes *adjacent* duplicate lines; `-c` prefixes each line with the number of repeats, `-d` prints only lines that are repeated. Because `uniq` compares a line only with the line just before it, it must almost always follow `sort`:

```
student@okan-lab:~/lab$ uniq -c data/cities.txt | head -n 4
      1 Antalya
      1 Trabzon
      1 Konya
      1 Bursa
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c
      2 Ankara
      6 Antalya
      5 Bursa
      5 Eskisehir
      1 Istanbul
      2 Izmir
      4 Konya
      5 Trabzon
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c | sort -rn | head -n 3
      6 Antalya
      5 Trabzon
      5 Eskisehir
student@okan-lab:~/lab$ cut -d, -f2 data/students.csv | sort | uniq -d
Elif Yilmaz
Mehmet Aydin
Onur Dogan
Selin Yildiz
```

On the unsorted file `uniq -c` reports `1 Antalya`, although Antalya occurs 6 times: the six lines are not next to each other. After `sort` the counts are right, and they add up: $2+6+5+5+1+2+4+5=30$ lines. The combination

```
sort | uniq -c | sort -rn
```

answers “how many of each, most frequent first” and is the most useful pipeline of this week. Equal counts (Trabzon and Eskisehir, 5 each) are ordered by the rest of the line, reversed by `-r`. Finally, `uniq -d` shows that four names occur twice in `students.csv`: different students (different ids) who happen to share a name.

7 Columns, Counts and Characters: `cut`, `wc`, `tr`

7.1 `cut`

```
student@okan-lab:~/lab$ head -n 3 data/students.csv | cut -d, -f2
name
Elif Yilmaz
Zeynep Dogan
student@okan-lab:~/lab$ head -n 3 data/students.csv | cut -d, -f2,5
name,final
Elif Yilmaz,99
Zeynep Dogan,36
student@okan-lab:~/lab$ head -n 2 logs/access.log | cut -d" " -f1,7,9
10.0.1.10 /about.html 404
10.0.1.10 /api/grades 200
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -c12-16
09:00
09:11
09:22
```

```
student@okan-lab:~/lab$ head -n 3 logs/system.log | cut -d" " -f4



student@okan-lab:~/lab$ head -n 3 logs/system.log | tr -s " " | cut -d" " -f4-
network link up
high memory usage
high memory usage
```

`cut -d SEP -f LIST` splits each line at the single character SEP and keeps the listed fields (counted from 1): `-f2,5` fields 2 and 5, `-f2-4` fields 2 to 4, `-f4-` field 4 to the end. `cut -c 12-16` keeps characters 12 to 16, useful for fixed-width columns.

In `system.log` the level is padded to five characters (`INFO␣␣network`), so between `INFO` and the message there are *two* spaces. `cut` treats each space as a separator, sees an empty field between them, and field 4 is empty — hence the three empty lines. `tr -s " "` squeezes runs of spaces into one space first.

7.2 `wc`

```
student@okan-lab:~/lab$ wc logs/system.log
 120  669 5207 logs/system.log
student@okan-lab:~/lab$ wc -l data/*
  30 data/cities.txt
  20 data/numbers.txt
  41 data/students.csv
  91 total
```

wc prints the number of lines, words and bytes: `system.log` has 120 lines, 669 words and 5207 bytes. `-l`, `-w` and `-c` select one count; with several files a `total` line is added. The line count is the number of newline characters, so a last line without a newline is not counted.

7.3 `tr`

```
student@okan-lab:~/lab$ head -n 2 docs/linux_quotes.txt | tr a-z A-Z
TALK IS CHEAP. SHOW ME THE CODE.
GIVEN ENOUGH EYEBALLS, ALL BUGS ARE SHALLOW.
student@okan-lab:~/lab$ head -n 2 data/students.csv | tr "," ";"
id;name;department;midterm;final
2026001;Elif Yilmaz;ECT;96;99
student@okan-lab:~/lab$ echo "Hello, World!" | tr -d ",!"
Hello World
```

`tr SET1 SET2` replaces every character of SET1 by the corresponding character of SET2; `tr -d SET` deletes characters; `tr -s SET` squeezes repeats. `tr` works on single characters, not words (`tr "cat" "dog"` replaces c by d, a by o and t by g everywhere), and it reads only standard input: it never takes a file name, so use < or a pipe.

8 More Tools: `head`, `tail`, `tee`, `find`, `xargs`

8.1 `head` and `tail` in pipelines; `tee`

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | head -n 2
2026001,Elif Yilmaz,ECT,96,99
2026002,Zeynep Dogan,ME,95,36
student@okan-lab:~/lab$ head -n 120 logs/access.log | tail -n 1
10.0.3.24 - - [12/Oct/2026:12:53:47 +0300] "GET /index.html HTTP/1.1" 404 4270
student@okan-lab:~/lab$ grep " 500 " logs/access.log | tee errors_500.txt | wc -l
24
student@okan-lab:~/lab$ head -n 2 errors_500.txt
10.0.5.38 - - [12/Oct/2026:08:14:26 +0300] "GET /index.html HTTP/1.1" 500 4583
10.0.5.38 - - [12/Oct/2026:08:21:39 +0300] "GET /images/logo.png HTTP/1.1" 500 4233
student@okan-lab:~/lab$ wc -l errors_500.txt
24 errors_500.txt
```

`head -n N` keeps the first N lines, `tail -n N` the last N, and `tail -n +N` everything from line N on. Combined, `head -n 120 | tail -n 1` extracts exactly line 120. `tail -f file` keeps the file open and prints new lines as they are appended: the standard way to watch a log live (stop with Ctrl+C).

`tee file` is a T-junction (named after the plumber's T-piece): it copies its standard input both to the file and to its standard output, so a pipeline can save an intermediate result and continue. `tee -a` appends. A link to Week 11: `sudo echo text > /etc/file` fails, because the redirection is performed by *your* shell, without root rights, before `sudo` even starts; `echo text | sudo tee /etc/file` works because the file is opened by `tee`, which runs as root.

8.2 `find`

`find WHERE TESTS` walks the whole directory tree below `WHERE` and prints every path that passes all tests.

```
student@okan-lab:~/lab$ find docs -name "*.txt"
docs/notes_02.txt
docs/syllabus.txt
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab$ find . -type d -name src
./src
./projects/beta/src
./projects/alpha/src
student@okan-lab:~/lab$ find . -size +10k
./logs/access.log
student@okan-lab:~/lab$ find photos -iname "*.jp*g" | wc -l
14
student@okan-lab:~/lab$ touch -d "2026-09-01" docs/notes_01.txt
student@okan-lab:~/lab$ find docs -mtime +30
docs/notes_01.txt
```

Test	Meaning
`-name "*.txt"`	the name matches the wildcard pattern (keep the quotes!)
`-iname`	the same, ignoring case
`-type f`, `-type d`	regular file, directory
`-size +10k`, `-size -10k`	larger / smaller than 10 KiB $= 10\,240$ bytes
`-mtime +30`, `-mtime -1`	modified more than 30 days ago / within the last 24 hours

Without the quotes in `-name "*.txt"`, the shell would expand `*.txt` in the *current* directory before `find` starts (Week 10). Only `access.log` ($23\,604$ bytes) exceeds 10 KiB. The 14 photos are the twelve `img0NN.jpg`, `IMG_014.JPG` and `holiday.jpeg`; `img013.png` does not match. Since all lab files were just created, we used `touch -d` to give `notes_01.txt` an old modification time (Week 10) so that `-mtime +30` finds something. Like `grep -r`, `find` lists entries in directory order; pipe into `sort` if you need a fixed order.

Errors from `find`. When `find` meets a directory it may not read (Week 11), it reports an error — on standard error, so it can be discarded separately:

```
student@okan-lab:~/lab$ mkdir secret && chmod 000 secret
student@okan-lab:~/lab$ find . -name "*.md"
./projects/beta/README.md
./projects/alpha/README.md
find: ‘./’secret: Permission denied
student@okan-lab:~/lab$ find . -name "*.md" 2> /dev/null
./projects/beta/README.md
./projects/alpha/README.md
```

(We removed the permissions of `secret` with `chmod 000`; afterwards, restore them with `chmod 755 secret` and remove it with `rmdir secret`.)

8.3 `-exec` and `xargs`

```
student@okan-lab:~/lab$ find . -name "*.c" -exec wc -l {} \;
7 ./src/hello.c
1 ./projects/beta/src/main.c
1 ./projects/alpha/src/main.c
student@okan-lab:~/lab$ find . -name "*.c" | xargs wc -l
  7 ./src/hello.c
  1 ./projects/beta/src/main.c
  1 ./projects/alpha/src/main.c
  9 total
student@okan-lab:~/lab$ find docs -name "*.txt" | xargs wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
wc: docs/final: No such file or directory
wc: report.txt: No such file or directory
 23 total
```

```
student@okan-lab:~/lab$ find docs -name "*.txt" -print0 | xargs -0 wc -l
  1 docs/notes_02.txt
 15 docs/syllabus.txt
  1 docs/notes_01.txt
  1 docs/notes_03.txt
  5 docs/linux_quotes.txt
  2 docs/final report.txt
 25 total
```

- `-exec cmd {} \;` runs `cmd` once for every file found, with `{}` replaced by its name. The `\;` marks the end of the command (the backslash stops the shell from interpreting `;`).
- `xargs cmd` reads words from its standard input and passes them to `cmd` as *arguments*: one `wc` receives all three names and prints a total. This is the difference between data and arguments: `find … | wc -l` would count the *names*; `find … | xargs wc -l` counts the lines *in the files*.
- By default `xargs` splits at spaces, so `final report.txt` became two non-existent names, `docs/final` and `report.txt`, and the total (23) missed its 2 lines. `find -print0` ends each name with a zero byte (which cannot occur in a file name) and `xargs -0` splits only there: total 25.

Safety: look before you delete

`find … -exec rm {} \;` and `find … | xargs rm` delete every file in the list, without asking and without a recycle bin. Always run the same command *without* `rm` first, read the list, and only then add `rm`. The same applies to `rm -r`.

9 Real Analyses with One-Line Pipelines

9.1 Know your data

Before writing a pipeline, look at a few lines (`head -n 3`) and decide how the fields are separated.

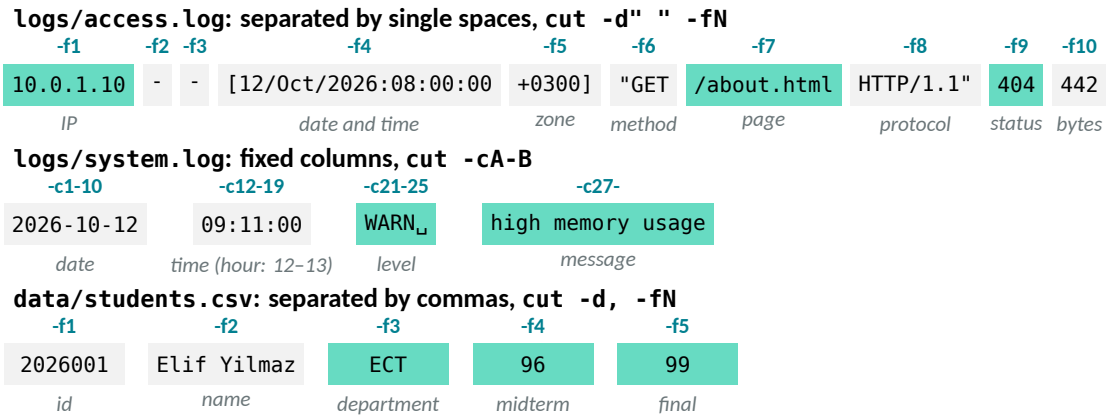


Figure 4: Where the fields are in the three lab files. Line 1 of `students.csv` is a header.

The web-server log uses the widespread *common log format*. Field 4 contains date and time; to get the hour you can also cut at colons: `cut -d: -f2` gives `08`. HTTP status codes form five classes: `2xx` success (`200` OK), `3xx` redirection (`301` moved permanently, `304` not modified: use your cached copy), `4xx` client

errors (403 forbidden, 404 not found) and 5xx server errors (500 internal server error).

9.2 The method: one stage at a time

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | head -n 3
404
200
500
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | head -n 3
200
200
200
```

```
student@okan-lab:~/lab$ cut -d" " -f9 logs/access.log | sort | uniq -c | sort -rn
    148 200
     57 404
     34 304
     24 500
     20 301
     17 403
```

Every counting question follows the same four steps:

1. **Extract** the column you need (`cut`), possibly after **filtering** the relevant lines (`grep`); check with `head`.
2. **Group**: `sort` brings equal values together.
3. **Count**: `uniq -c`.
4. **Rank**: `sort -rn`, then `head -n N` for a top N.

Never type a six-stage pipeline in one go: press the up-arrow, add one stage, look at the result, repeat. Check results whenever you can. Here the counts add up to the number of lines: $148 + 57 + 34 + 24 + 20 + 17 = 300$, and the share of successful requests is $148/300 \approx 49.3\%$.

9.3 access.log: pages, visitors, broken links

```
student@okan-lab:~/lab$ cut -d" " -f7 logs/access.log | sort | uniq -c | sort -rn | head -n 5
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
     33 /images/logo.png
     32 /api/grades
student@okan-lab:~/lab$ cut -d" " -f1 logs/access.log | sort | uniq -c | sort -rn | head -n 3
     58 10.0.2.17
     56 10.0.4.31
     52 10.0.6.45
student@okan-lab:~/lab$ grep " 404 " logs/access.log | cut -d" " -f7 | sort | uniq -c | sort -
    rn | head -n 3
      9 /images/logo.png
      7 /index.html
      7 /about.html
```

The same recipe with field 7 (page) and field 1 (IP address). The third pipeline first filters the 404 lines and

then extracts the page: the most frequent broken link is `/images/logo.png` (9 times). The spaces in `" 404 "` matter: without them, a byte count such as `4042` or an IP part could also match. Ties are common in small data: `/index.html` and `/images/logo.png` both have 33 requests.

9.4 system.log and students.csv

```
student@okan-lab:~/lab$ grep -c ERROR logs/system.log
13
student@okan-lab:~/lab$ grep ERROR logs/system.log | cut -c12-13 | uniq -c
      3 09
      4 10
      1 12
      3 13
      2 14
student@okan-lab:~/lab$ grep WARN logs/system.log | cut -c27- | sort | uniq -c
     11 high memory usage
     17 temperature 71C
student@okan-lab:~/lab$ tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c | sort -rn
     17 ECT
     11 ME
     10 CS
      2 EE
student@okan-lab:~/lab$ tail -n +2 data/students.csv | sort -t, -k5 -nr | head -n 3 | cut -d, -
    f2,3,5
Onur Dogan,ME,100
Elif Yilmaz,ECT,99
Emre Dogan,ME,98
student@okan-lab:~/lab$ grep ",ECT," data/students.csv | sort -t, -k4 -nr | head -n 3
2026017,Ipek Yilmaz,ECT,100,36
2026012,Arda Celik,ECT,98,63
2026006,Mehmet Aydin,ECT,98,85
```

- **Errors per hour.** Here `uniq -c` works without `sort`, because the log is already in hour order, so equal hours are adjacent. Hours without errors (11 and 15) are simply missing: `uniq` cannot count what is not there. Check: $3 + 4 + 1 + 3 + 2 = 13$.
- **Warnings.** `cut -c27-` keeps the message; $11 + 17 = 28$ WARN lines.
- **Departments.** $17 + 11 + 10 + 2 = 40$ students $= 41$ lines minus the header.
- **Best final grades.** Sort whole lines by field 5 first, cut the display columns last; cutting first would lose the column you sort by.

10 Command Reference

Command / syntax	Meaning
`cmd > f`	standard output to `f` (create or empty first)
`cmd >> f`	standard output appended to `f`
`cmd < f`	standard input from `f`
`cmd 2> f`, `2>> f`	standard error to `f` (overwrite, append)
`cmd > f 2>&1`, `cmd &> f`	both streams to `f`

Command / syntax	Meaning
`cmd 2> /dev/null`	discard error messages
`cmd1 | cmd2`	standard output of `cmd1` to standard input of `cmd2`
`set -o noclobber`	> refuses to overwrite existing files (>| forces)
`grep PAT files`	lines containing the regular expression PAT
`-i -n -v -c -w -r -l`	ignore case, line numbers, invert, count, whole word, recursive, file names only
`sort`	sort lines; `-n` numeric, `-r` reverse, `-t,` separator, `-k5` key field, `-u` unique, `-o f` output file
`uniq`	collapse adjacent repeats; `-c` count, `-d` only repeated
`cut -d, -f2,5`	fields 2 and 5 of comma-separated lines; `-f4-` field 4 to the end
`cut -c12-16`	characters 12 to 16
`wc`	lines, words, bytes; `-l`, `-w`, `-c`
`tr a-z A-Z`	translate characters (reads stdin only); `-d` delete, `-s` squeeze
`head -n N`, `tail -n N`	first / last N lines
`tail -n +N`	from line N to the end (skip a header with $N = 2$)
`tail -f f`	follow a growing file (Ctrl+C stops)
`tee f`, `tee -a f`	copy stdin to `f` and to stdout
`find DIR -name "*.c"`	files below `DIR` whose name matches; `-iname` ignores case
`-type f|d, -size +10k, -mtime +30`	file type, size, age in days
`-exec cmd {} \;`	run `cmd` on each file found
`-print0 | xargs -0 cmd`	pass names (even with spaces) as arguments to `cmd`
`strace -f -e trace=… cmd`	show the system calls of `cmd` and its children

11 Common Errors and What They Mean

All of the following were produced on purpose (`code/demo_mistakes.sh`; the 15 lines printed by the unquoted `grep` are shortened):

```
student@okan-lab:~/lab$ cp data/cities.txt c.txt
student@okan-lab:~/lab$ sort c.txt > c.txt
student@okan-lab:~/lab$ wc -l c.txt
0 c.txt
student@okan-lab:~/lab$ grep user login logs/system.log
grep: login: No such file or directory
logs/system.log:2026-10-12 09:44:00 INFO  user login
...
logs/system.log:2026-10-12 14:43:00 INFO  user login
logs/system.log:2026-10-12 14:27:00 INFO  user login
student@okan-lab:~/lab$ grep "user login" logs/system.log | wc -l
15
student@okan-lab:~/lab$ ls data > wc
student@okan-lab:~/lab$ ls
README.txt  c.txt  docs  photos    scripts  wc
backup      data   logs  projects  src
student@okan-lab:~/lab$ cat wc
cities.txt
numbers.txt
students.csv
```

```
student@okan-lab:~/lab$ sort -t, -k5 -n data/students.csv | head -n 2
id,name,department,midterm,final
2026032,Onur Dogan,ECT,41,35
```

`sort c.txt > c.txt` leaves an empty file. The shell performs the redirection *first*: it truncates `c.txt` to 0 bytes, then starts `sort`, which reads an empty file. The same happens with any `cmd f > f`. Write to a new file and rename it (`sort c.txt > c2.txt && mv c2.txt c.txt`), or use `sort -o c.txt c.txt`.

`grep: login: No such file or directory`. Without quotes, the command `grep user login logs/system.log` searches for `user` in two files, `login` and `logs/system.log`. Quote patterns that contain spaces: `grep "user login" logs/system.log`.

A file called `wc` appears. `ls data > wc` redirects into a file named `wc`; it does not run `wc`. Use the pipe: `ls data | wc -l`.

The header line appears in the result. `id,name,…` is not a number, so `sort -n` treats it as 0 and places it first. Skip it with `tail -n +2`.

`uniq -c` gives counts of 1. The input was not sorted; use `sort | uniq -c`.

Errors still appear although you wrote `2>&1`. The order was `2>&1 > f`; write `> f 2>&1` (Figure 2).

`cut` prints empty lines. The separator occurs twice in a row (padded columns); squeeze first with `tr -s " "` or use `cut -c`.

`xargs: No such file or directory` for half a name. A file name contains a space; use `find … -print0 | xargs -0 …`.

`Permission denied` lines from `find`. Directories you may not read (Week 11). If they do not matter, add `2> /dev/null`.

A pipeline seems to hang. A command without a file argument waits for input from the keyboard (e.g. `sort` alone, or `grep ERROR` without a file). Press Ctrl+D (end of input) or Ctrl+C.

`tr: extra operand`. `tr` does not take file names; use `tr a-z A-Z < file`.

12 Lab Sheet

Preparation: `bash mcmp117_lab_setup.sh` then `cd ~/lab`. Write one command line per exercise (a pipeline counts as one line) and its output. Points in brackets, 100 in total. Solutions are in Section 17; try first!

Part A --- Redirection (20 points)

A1 [4] Save the list of files in `data/` into `list.txt`, then append the list of `docs/` to the same file. How many lines does `list.txt` have?

A2 [4] Run `ls data nosuch` so that the listing goes to `ok.txt`, the error message to `err.txt`, and nothing appears on the screen.

A3 [4] Print only the number of lines of `logs/system.log`, without the file name.

A4 [4] Predict, then check: which of `ls data nosuch > both.txt 2>&1` and `ls data nosuch 2>&1 > both.txt` prints something on the screen, and why?

A5 [4] Create an unreadable directory with `mkdir secret && chmod 000 secret`. The command `find . -name "*.md"` now prints an error. Change it so that only the results appear.

Part B --- grep (22 points)

B1 [4] How many ERROR lines does `logs/system.log` contain?

B2 [4] Show the first 3 lines of `system.log` that are *not* INFO lines, with their line numbers.

B3 [4] How many requests in `access.log` received the status code 404?

B4 [4] Which students belong to the EE department?

B5 [6] How many requests came from the IP address `10.0.5.38`? Make sure the dots cannot match other characters and that only the first field can match.

Part C --- One-line pipelines (36 points, 6 each)

C1 How many students are there in each department (without the header)?

C2 Print the names and final grades of the 3 students with the highest final grade.

C3 Count the requests per status code, most frequent first.

C4 Which are the 3 most requested pages?

C5 Which page produced the most 404 errors, and how many?

C6 How many ERROR lines does `system.log` contain in each hour?

Part D --- Challenge (22 points)

D1 [5] Which student names appear more than once in `students.csv`?

D2 [5] In one command line, save all lines with status 500 into `errors_500.txt` *and* print how many there are.

D3 [6] Count the lines of all `.txt` files below `docs/`, including `final report.txt`, with `find` and `xargs`. What is the total?

D4 [6] How many different IP addresses appear in `access.log`? Which address made the fewest requests, and how many?

13 Summary

Week 12 in seven sentences

1. Every process starts with three open file descriptors: 0 stdin, 1 stdout, 2 stderr — the same descriptor 1 that `write(1, …)` used in Weeks 2–3.

2. Redirection (`> >> < 2> 2>&1 &> /dev/null`) changes where these descriptors point; the shell does it with `open()` and `dup2()` between `fork()` and `exec()`, so the program never knows.
3. Redirections are applied left to right: `> f 2>&1` captures both streams, `2>&1 > f` does not.
4. A pipe `cmd1 | cmd2`, created with `pipe()`, connects two concurrently running processes through a kernel buffer that behaves like the bounded buffer of Week 5.
5. Filters read files or stdin and write stdout: `grep` (with `^ $ . [ ] *`), `sort`, `uniq`, `cut`, `wc`, `tr`, `head`, `tail`, `tee`.
6. `find` searches the tree by name, type, size and age; `-exec` or `xargs` act on the results, `-print0 | xargs -0` for names with spaces.
7. Counting questions follow one recipe: extract → `sort` → `uniq -c` → `sort -rn` → `head`, built one stage at a time and checked against totals.

14 Glossary

File descriptor Small integer that indexes a process's table of open files; 0, 1, 2 are the standard streams.

Standard input / output / error The streams on descriptors 0, 1, 2; by default the keyboard and the screen.

Redirection Making a standard stream refer to a file instead of the terminal (>, >>, <, 2>).

`dup2(old, new)` System call that makes descriptor `new` a copy of `old`.

Pipe Kernel buffer with a read end and a write end, connecting the output of one process to the input of another.

Pipeline Commands connected by pipes, running concurrently.

`SIGPIPE` Signal sent to a process that writes to a pipe with no reader; it terminates the process by default.

Filter Program that reads its input from files or stdin and writes its result to stdout.

Regular expression Pattern language for describing sets of strings, used by `grep`.

`/dev/null` Device file that discards everything written to it.

15 Try It Yourself

More experiments

1. Run `ls -l /proc/self/fd | cat`, then `ls -l /proc/self/fd 2>&1 | cat`. Which descriptors are pipes in each case?
2. Run `ls docs` and `ls docs | cat`. Why does the first print `'final report.txt'` with quotes and in columns, but the second not?
3. Use `strace -f -o t.txt -e trace=pipe2,dup2,execve bash -c "..."` on a three-

stage pipeline of your own. How many `pipe2()` calls do you see? (One less than the number of stages.)

4. In one terminal run `tail -f logs/system.log`; in a second terminal run `echo "2026-10-12 15:00:00 ERROR test" >> logs/system.log`. What happens in the first terminal?
5. Find the hour of the day with the most requests in `access.log` (hint: `cut -d: -f2`). Why is the answer surprising for this generated file?

16 Review Questions

Conceptual

1. What are standard input, standard output and standard error? Which file descriptors do they use, and why is standard error separate from standard output?
2. Explain step by step (system calls) what the shell does to run `sort data/cities.txt > sorted.txt`.
3. Why does `cmd 2>&1 > f` still print error messages on the screen?
4. Explain why `fork()` and `exec()` being separate calls makes redirection easy.
5. A pipe is a bounded buffer. What happens when the writer is faster than the reader? When the reader exits early?
6. Why must the input of `uniq -c` normally be sorted?
7. What is the difference between `find docs -name "*.txt" | wc -l` and `find docs -name "*.txt" | xargs wc -l`?
8. Explain the different meanings of `*` in `ls *.txt` and in `grep "[0-9]*"`.

Numerical

9. A pipe buffer holds 16 pages of 4096 bytes. How many lines of the form `MCMP117\n` (8 bytes each) fit into it before the writer blocks?
10. In `access.log`, 148 of 300 requests returned 200 and 57 returned 404. What percentage of requests were client errors (4xx), given that 17 more returned 403?
11. `grep -c "5.38"` counts 39 lines, `grep -c "5\.38"` 37. How many lines matched only because of the unescaped dot?
12. A pipeline has k stages. How many processes, `pipe()` calls and `dup2()` calls does the shell need at least (ignoring redirections to files)?

Answers to the numerical questions

9. $16 \times 4096 = 65\,536$ bytes; $65\,536/8 = 8192$ lines.
10. $57 + 17 = 74$ client errors; $74/300 \approx 24.7\%$.

11. $39 - 37 = 2$ lines (the time `15:38` and the byte count 5438).

12. k processes, $k-1$ pipes, and $2(k-1)$ calls of `dup2()`: every pipe needs one `dup2()` in its writer (onto descriptor 1) and one in its reader (onto descriptor 0). For $k = 2$: 2 processes, 1 pipe, 2 `dup2()` calls, exactly as in the trace of Section 4.

17 Lab Sheet Solutions

All solutions were run in a freshly created lab; the script `code/lab_solutions.sh` runs them all, and its complete real output is in `code/out_lab_solutions.txt`. Outputs that already appeared in the tutorial sections are summarised. Other correct solutions exist; what counts is the right output and a sensible method.

	Command line	Expected output / answer

A1	`ls data > list.txt`
	`ls docs >> list.txt`
	`wc -l list.txt`
		9 `list.txt`: 3 names from `data/` plus 6 visible names from `docs/` (`.settings` is hidden).

A2	`ls data nosuch > ok.txt 2> err.txt` `cat err.txt`	Nothing on the screen; `err.txt` contains `ls: cannot access 'nosuch': No such file or directory`.
A3	`wc -l < logs/system.log`	`120`
A4	`ls data nosuch > both.txt 2>&1` `ls data nosuch 2>&1 > both.txt`	Only the second prints the error: `2>&1` copies descriptor 1 while it still points to the terminal (Figure 2).
A5	`find . -name "*.md" 2> /dev/null`	`./projects/beta/README.md` and `./projects/alpha/README.md`, no error line.
B1	`grep -c ERROR logs/system.log`	13
B2	`grep -vn INFO logs/system.log | head -n 3`	Lines 2, 3 (WARN high memory usage) and 13 (ERROR failed to mount /dev/sdb1).
B3	`grep -c " 404 " logs/access.log`	57
B4	`grep ",EE," data/students.csv`	Deniz Aydin and Ece Ozturk (`2026011`, `2026031`).
B5	`grep -c "^10\.0\.5\.38 " logs/access.log`	37. Escaped dots, anchored at the line start and followed by a space, so neither `15:38`, `5438` nor `10.0.5.380` can match.
C1	`tail -n +2 data/students.csv | cut -d, -f3 | sort | uniq -c`	CS 10, ECT 17, EE 2, ME 11.
C2	`tail -n +2 data/students.csv | sort -t, -k5 -nr | head -n 3 | cut -d, -f2,5`	Onur Dogan 100, Elif Yilmaz 99, Emre Dogan 98.
C3	`cut -d" " -f9 logs/access.log | sort | uniq -c | sort -rn`	200: 148, 404: 57, 304: 34, 500: 24, 301: 20, 403: 17 (sum 300).
C4	`cut -d" " -f7 logs/access.log | sort | uniq -c | sort -rn | head -n 3`	`/courses/mcmp117` 41, `/about.html` 37, `/index.html` 33 — tied with `/images/logo.png` (33); either is accepted if the tie is noticed.

	Command line	Expected output / answer
C5	`grep " 404 " logs/access.log | cut -d" " -f7 | sort | uniq -c | sort -rn | head -n 1`	`/images/logo.png`, 9 times.
C6	`grep ERROR logs/system.log | cut -c12-13 | uniq -c`	09: 3, 10: 4, 12: 1, 13: 3, 14: 2 (hours 11 and 15: none; total 13).
D1	`cut -d, -f2 data/students.csv | sort | uniq -d`	Elif Yilmaz, Mehmet Aydin, Onur Dogan, Selin Yildiz. The header cannot be a duplicate, so it need not be removed.
D2	`grep " 500 " logs/access.log | tee errors_500.txt | wc -l`	24; `errors_500.txt` holds the 24 lines.
D3	`find docs -name "*.txt" -print0 | xargs -0 wc -l`	Total 25 lines (with plain `xargs` the name with a space breaks and the total is 23).
D4	`cut -d" " -f1 logs/access.log | sort -u | wc -l` `cut -d" " -f1 logs/access.log | sort | uniq -c | sort -n | head -n 1`	6 addresses; `10.0.5.38` made the fewest requests, 37.

Grading hints. Full points for a working single command line with the correct output; half points if the output is right but obtained with several commands or temporary files (except where the exercise asks for files); no points for a correct number obtained by counting by hand.

18 Looking Ahead: Week 13

- **Week 13: Processes & Job Control.** Today the shell created processes for us and connected them with pipes. Next week we watch processes from the outside: `ps`, `pstree` and `top` (PIDs, parent PIDs and the process states of Week 3), foreground and background jobs (&, `jobs`, `fg`, `bg`, Ctrl+Z, Ctrl+C), signals (`kill`, `SIGTERM` versus `SIGKILL` — and the `SIGPIPE` that stopped `yes` today), priorities with `nice` (Week 5), and a zombie process you can see with your own eyes.
- **Week 14:** shell scripting, where today's pipelines become the building blocks of automated reports.

Further reading (optional)

- A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed., Wiley: Section 2.2 (user interfaces, the command interpreter), Section 3.3 (process creation with `fork()` and `exec()`) and Section 3.7.4 (Pipes).
- R. H. and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*:
  - Chapter 5, "Interlude: Process API" (Section 5.4 shows how a shell redirects output between `fork()` and `exec()`): https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-api.pdf
  - Chapter 39, "Interlude: Files and Directories" (file descriptors, `dup()`): https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf
- GNU Bash manual, "Redirections": https://www.gnu.org/software/bash/manual/html_node/Redirections.html

- GNU Coreutils manual (`sort`, `uniq`, `cut`, `tr`, `wc`, `head`, `tail`, `tee`): https://www.gnu.org/software/coreutils/manual/coreutils.html